Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет програмних систем та прикладної математики
Кафедра програмного забезпечення комп'ютерних систем

Лабораторна робота №1

з дисципліни «Основи вебпрограмування та розроблення мобільних застосунків»

на тему «Розроблення застосунку мовою Java»

Виконала	Прийняв
Студентка 2-го курсу	«____» «_____» 2026 р.
групи КП-43	д-р філософії, асистент
Тетяна СТРІЛЕЦЬКА	Андрій СЕВЕРІН
	(ім’я ПРІЗВИЩЕ)
(ім’я ПРІЗВИЩЕ)	

Нараховані бали:

Якість виконання роботи (7)	Відповідь під час захисту (5)	Своєчасність представлення роботи (2)

Оцінка:

Київ 2026

Мета: формування навичок розроблення застосунків мовою Java.

Завдання: розробити консольний застосунок відповідно до варіанту студента.

Порядок виконання роботи

1. Обрати варіант з предметною областю для застосунку, що розроблятиметься.
2. Створити Git-репозиторій для роботи над завданням лабораторної роботи (можна використовувати будь-який сервіс, наприклад, Github, BitBucket тощо). Додати *.gitignore* файл.
3. Створити проєкт використовуючи систему збірки проєктів Maven або Gradle.
4. Визначити та реалізувати ключові сутності та бізнес логіку застосунку.
    - Реалізувати принаймні 3 класи з предметної області.
    - Кожен клас має містити принаймні один метод з бізнес логікою (для якого можна буде написати юніт тести).
    - У класах реалізувати геттери / сеттери, методи *equals* та *hashCode*.
    - Методи, що реалізовують логіку застосунку мають бути незалежними від способу виводу інформації користувачеві.
5. Написати модульні тести (принаймні 9). Вони мають покривати випадки, що викликають різні гілки коду, зокрема, при використанні різних вхідних даних (в тому числі невалідних), обробці виняткових ситуацій тощо.
6. Реалізувати експорт та імпорт даних (об’єктів предметної області) з використанням файлів (формат довільний; наприклад, csv, xml, json). Передбачити можливість сортування даних за певним критерієм перед експортом. Написати юніт тести для експорту та імпорту даних з використанням мок-об’єктів.
7. Реалізувати консольне меню для демонстрації роботи застосунку.
8. Сформулювати висновки.

1. Формулювання обраного завдання

- **Предметна галузь**

Назва: Система автоматизації бронювання та управління готельним фондом.
Опис: Застосунок призначений для цифровізації процесів роботи готелю, що включає управління номерним фондом, реєстрацію відвідувачів та проведення операцій бронювання з автоматичним розрахунком фінансових показників та обліком доступності місць.

- **Ключові сутності (Класи)**

Для реалізації бізнес-логіки згідно з варіантом завдання було виділено такі класи:

1. **Hotel (HotelService)** — центральний керуючий клас, що містить логіку управління готелем, списки номерів та історію бронювань.
2. **Room (Номер)** — сутність, що описує характеристики житлової одиниці (номер, ціна за добу, статус доступності).
3. **Visitor (Відвідувач)** — модель даних клієнта (ID, ПІБ, контактні дані).
4. **Booking (Бронювання)** — об’єкт, що пов’язує відвідувача з номером на певний термін та фіксує параметри угоди.

2. Функціональні вимоги до застосунку

У ході розробки було реалізовано наступний функціонал:

Процес бронювання:

- Перевірка доступності обраного номера на вказаний період.
- Автоматичне обчислення загальної суми до сплати на основі тарифу номера та кількості ночей.
- Динамічне оновлення лічильників доступних та заброньованих номерів у системі.

Звітність та статистика:

- Виведення актуальної статистики щодо наповненості готелю.
- Сортування доступних номерів за їхнім номером у порядку зростання перед виводом на екран та експортом у JSON-файл.

Управління даними (CRUD операції):

- Реалізовано можливість додавання нових номерів до бази готелю.
- Перегляд характеристик (номер, ціна) та поточного стану доступності.
- Автоматичне оновлення статусу номера (вільний/зайнятий) у процесі бронювання або звільнення.
- Створення нових профілів клієнтів з автоматичною генерацією унікального випадкового ID (за допомогою ThreadLocalRandom).
- Валідація вхідних даних: реалізовано перевірку формату електронної пошти (email) на коректність перед реєстрацією.

Робота з файлами:

- Експорт поточного стану системи у формат JSON (з використанням бібліотеки Jackson).
- Імпорт збережених даних для відновлення роботи застосунку після перезапуску.

3. Посилання на Git репозиторій

https://github.com/str-tan/striletska-kp-43-lab1-v9-hotel

4. Діаграма розроблених класів

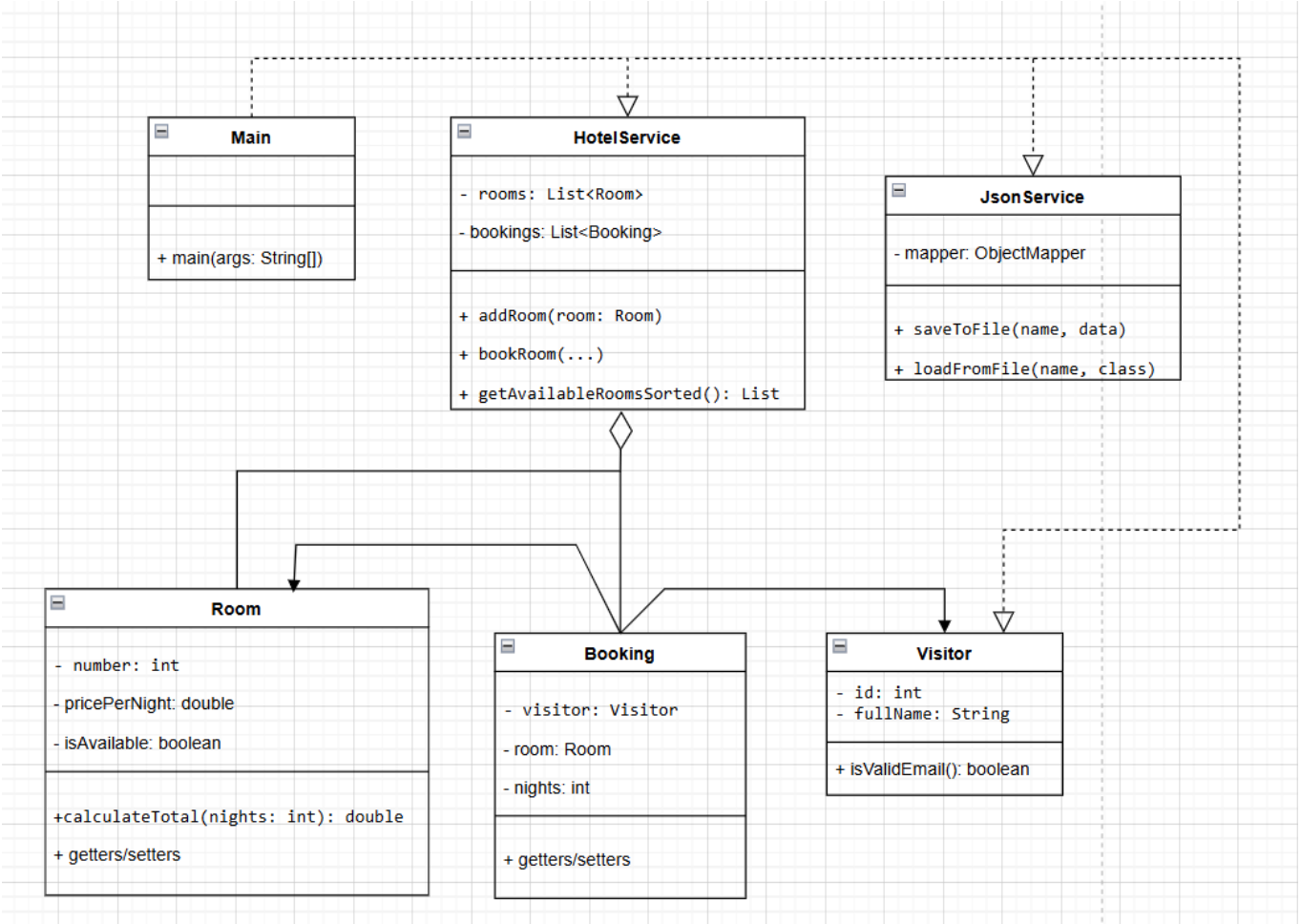

Рис.1 - Діаграма розроблених класів

5. Лістинг конфігурації системи збірки проєктів

Файл: pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <groupId>com.hotel</groupId>
```

```
    <artifactId>HotelApp</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>

<maven.compiler.source>17</maven.compiler.source>

<maven.compiler.target>17</maven.compiler.target>

<project.build.sourceEncoding>UTF-8</project.build.sou
rceEncoding>
    </properties>

    <dependencies>
        <dependency>

<groupId>com.fasterxml.jackson.core</groupId>
            <artifactId>jackson-databind</artifactId>
            <version>2.16.1</version>
        </dependency>

        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.mockito</groupId>
```

```
            <artifactId>mockito-core</artifactId>
            <version>5.5.0</version>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.mockito</groupId>

<artifactId>mockito-junit-jupiter</artifactId>
            <version>5.5.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

</project>
```

2. Лістинг коду

Моделі (пакет com.hotel.model**):**

- **Room.java:**

```
package com.hotel.model;

import java.util.Objects;

public class Room {
    private int number;
    private double pricePerNight;
    private boolean isAvailable;

    public Room() {}//для json
```

```java
    public Room(int number, double pricePerNight) {
        this.number = number;
        this.pricePerNight = pricePerNight;
        this.isAvailable = true;
    }

    public void setAvailable(boolean available) {
        this.isAvailable = available;
    }

    public int getNumber() { return number; }
    public double getPricePerNight() { return
pricePerNight; }
    public boolean isAvailable() { return
isAvailable; }

    public double calculateTotal(int nights) {
        if (nights <= 0) throw new
IllegalArgumentException("Кількість ночей має бути
> 0");
        return pricePerNight * nights;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() !=
o.getClass()) return false;
        Room room = (Room) o;
        return number == room.number;
```

```
    }

    @Override
    public int hashCode() { return
Objects.hash(number); }

    @Override
    public String toString() { return "Номер " +
number + " ($" + pricePerNight + ")"; }
}
```

- **Visitor.java:**

```
package com.hotel.model;

import java.util.Objects;
import java.util.concurrent.ThreadLocalRandom;

public class Visitor {
    private int id;
    private String fullName;
    private String email;

    public Visitor() {}//для json

    public Visitor(String fullName, String email) {
        this.id =
ThreadLocalRandom.current().nextInt(1000, 10000);
        this.fullName = fullName;
        this.email = email;
    }
```

```
    public int getId() { return id; }
    public String getFullName() { return fullName;
}
    public String getEmail() { return email; }

    public boolean isValidEmail() {
        return email != null && email.contains("@")
&& email.contains(".");
    }

    @Override
    public String toString() {
        return "Відвідувач: " + fullName + " (" +
email + ")";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() !=
o.getClass()) return false;
        Visitor visitor = (Visitor) o;
        return id == visitor.id;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
```

```
}
```

- **Booking.java:**

```
package com.hotel.model;

public class Booking {
    private Visitor visitor;
    private Room room;
    private int nights;
    private double totalAmount;

    public Booking() {}//для json

    public Booking(Visitor visitor, Room room, int
nights) {
        this.visitor = visitor;
        this.room = room;
        this.nights = nights;

        this.totalAmount =
room.calculateTotal(nights);
    }
    public int getNights() {
        return nights;
    }

    public Visitor getVisitor() { return visitor; }
    public Room getRoom() { return room; }
    public double getTotalAmount() { return
totalAmount; }
```

```
    @Override
    public String toString() {
        return "Бронювання: " +
visitor.getFullName() + " у номері " +
room.getNumber() +
                ". На " + nights + " ночей. До
сплати: $" + totalAmount;
    }
}
```

Сервіси (пакет com.hotel.service)**:**

- **HotelService.java:**

```
package com.hotel.service;

import com.hotel.model.*;
import java.util.*;
import java.util.stream.Collectors;

public class HotelService {
    private final List<Room> rooms = new
ArrayList<>();
    private final List<Booking> bookings = new
ArrayList<>();

    public void addRoom(Room room) {
rooms.add(room); }

    public List<Room> getAvailableRoomsSorted() {
        return rooms.stream()
                .filter(Room::isAvailable)
```

```
.sorted(Comparator.comparingInt(Room::getNumber))
                .collect(Collectors.toList());
    }

    public List<Booking> getBookingsSortedByRoom()
{
        return bookings.stream()
                // Показуємо лише ті бронювання, де
кімната досі ЗАЙНЯТА
                .filter(b ->
!b.getRoom().isAvailable())
                .sorted(Comparator.comparingInt(b
-> b.getRoom().getNumber()))
                .collect(Collectors.toList());
    }

    public int getOccupiedRoomsCount() {
        return (int) rooms.stream().filter(r ->
!r.isAvailable()).count();
    }
    public boolean releaseRoom(int roomNumber) {
        Optional<Room> roomOpt = rooms.stream()
                .filter(r -> r.getNumber() ==
roomNumber)
                .findFirst();

        if (roomOpt.isPresent()) {
            Room room = roomOpt.get();
            if (!room.isAvailable()) {
```

```
                room.setAvailable(true);
                // ВИДАЛЯЄМО бронювання з нашого
списку
                bookings.removeIf(b ->
b.getRoom().getNumber() == roomNumber);
                return true;
            }
        }
        return false;
    }

    public boolean isRoomAvailable(int roomNumber)
{
        return rooms.stream().anyMatch(r ->
r.getNumber() == roomNumber && r.isAvailable());
    }
    public Booking bookRoom(Visitor visitor, int
roomNumber, int nights) {
        Room room = rooms.stream()
                .filter(r -> r.getNumber() ==
roomNumber && r.isAvailable())
                .findFirst()
                .orElseThrow(() -> new
NoSuchElementException("Номер зайнято!"));

        Booking booking = new Booking(visitor,
room, nights);
        room.setAvailable(false);
        bookings.add(booking);
        return booking;
```

```
    }
}
```

- **JsonService.java:**

```
package com.hotel.service;

import
com.fasterxml.jackson.databind.ObjectMapper;
import
com.fasterxml.jackson.databind.SerializationFe
ature;
import java.io.File;
import java.io.IOException;
import java.util.List;

public class JsonService {
    private final ObjectMapper mapper;

    public JsonService() {
        this.mapper = new ObjectMapper();
        //JSON з відступами

this.mapper.enable(SerializationFeature.INDENT
_OUTPUT);
    }

    //Експорт даних у файл
    public <T> void saveToFile(String fileName,
List<T> data) throws IOException {
        mapper.writeValue(new File(fileName),
data);
```

```
    }

    //Імпорт даних із файлу
    public <T> List<T> loadFromFile(String
fileName, Class<T> clazz) throws IOException {
        File file = new File(fileName);
        if (!file.exists()) return List.of();

        return mapper.readValue(file,

mapper.getTypeFactory().constructCollectionTyp
e(List.class, clazz));
    }
}
```

Точка входу:

- **Main.java:**

```
package com.hotel;

import com.hotel.model.*;
import com.hotel.service.HotelService;
import com.hotel.service.JsonService;
import java.io.IOException;
import java.util.*;

public class Main {
    public static void main(String[] args) {
        HotelService hotelService = new
HotelService();
```

```
        JsonService jsonService = new
JsonService();
        Scanner scanner = new Scanner(System.in);

        hotelService.addRoom(new Room(101, 500.0));
        hotelService.addRoom(new Room(102,
1200.0));
        hotelService.addRoom(new Room(103, 800.0));
        hotelService.addRoom(new Room(75, 100.0));
        hotelService.addRoom(new Room(47, 200.0));

        boolean running = true;

        while (running) {
            try {
                System.out.println("\n=== ГОТЕЛЬ
'Star Rain' ===");
                showStatistics(hotelService);

                System.out.println("\n--- МЕНЮ
---");
                System.out.println("1. Забронювати
номер");
                System.out.println("2. Звільнити
номер");
                System.out.println("3. Переглянути
всі бронювання");
                System.out.println("4. Вийти");
                System.out.println("5.
Збереження");
```

```
System.out.println("6.Завантаження");
                System.out.print("Оберіть дію: ");

                int choice = scanner.nextInt();
                scanner.nextLine();

                switch (choice) {
                    case 1:

System.out.println("\nДоступні номери
(відсортовано):");

hotelService.getAvailableRoomsSorted().forEach(r
->

System.out.println("Номер " + r.getNumber() + " |
Ціна: $" + r.getPricePerNight() + " | Вільний: " +
r.isAvailable())
                        );

                        System.out.print("\nВведіть
номер кімнати: ");
                        int rNum =
scanner.nextInt();
                        scanner.nextLine();

                        if
(!hotelService.isRoomAvailable(rNum)) {
```

```
System.out.println("Помилка: Номер вже зайнято або
не існує!");
                            break;
                        }

                        System.out.print("Ваш
Email: ");
                        String email =
scanner.nextLine();
                        Visitor visitor = new
Visitor("Гість", email);

                        if
(!visitor.isValidEmail()) {

System.out.println("Помилка: Email (" +
visitor.getEmail() + ") некоректний!");
                            break;
                        }

                        System.out.print("Повне
ім'я: ");
                        String name =
scanner.nextLine();
                        visitor = new Visitor(name,
email);


System.out.println("Реєструємо клієнта: " +
```

```
visitor.getFullName() + " (ID: " + visitor.getId()
+ ")");

                        System.out.print("Скільки
ночей? ");
                        int nights =
scanner.nextInt();
                        scanner.nextLine();

                        Booking b =
hotelService.bookRoom(visitor, rNum, nights);

                        System.out.println(" " +
b.getVisitor().getFullName() + ", заброньовано на
" + nights + " ночей.");

System.out.println("Загальна сума: $" +
b.getTotalAmount());
                        break;

                    case 2:
                        System.out.print("Введіть
номер кімнати для звільнення: ");
                        int roomToFree =
scanner.nextInt();
                        scanner.nextLine();

                        if
(hotelService.releaseRoom(roomToFree)) {
```

```
System.out.println("Номер " + roomToFree + " тепер
знову вільний.");
                        } else {

System.out.println("!Увага: Номер " + roomToFree +
" і так був вільним.");
                        }
                        break;

                    case 3:
                        System.out.println("\n---
ДЕТАЛЬНИЙ ЗВІТ ---");
                        List<Booking>
activeBookings =
hotelService.getBookingsSortedByRoom();

                        if
(activeBookings.isEmpty()) {

System.out.println("Бронювань немає.");
                        } else {
                            for (Booking booking :
activeBookings) {
                                System.out.println(
                                        "Кімната №"
+ booking.getRoom().getNumber() +
                                                    " |
Клієнт: " + booking.getVisitor().getFullName() +
```

```
                                                    " |
Ночей: " + booking.getNights() +
                                                    " |
РАЗОМ: $" + booking.getTotalAmount()
                                    );
                                }

                                double totalRevenue =
activeBookings.stream()

.mapToDouble(Booking::getTotalAmount)
                                        .sum();

System.out.println("\nЗагальна виручка готелю: $"
+ totalRevenue);
                            }
                            break;

                        case 4:
                            running = false;
                            System.out.println("До
зустрічі!");
                            break;

                        case 5: // Збереження
                            try {
                                List<Room> roomsToSave
= hotelService.getAvailableRoomsSorted();
```

```
jsonService.saveToFile("rooms_data.json",
roomsToSave);

System.out.println("Дані успішно збережено в
rooms_data.json");
                        } catch (IOException e) {

System.out.println("Помилка запису: " +
e.getMessage());
                        }
                        break;

                    case 6: // Завантаження
                        try {
                            List<Room> imported =
jsonService.loadFromFile("rooms_data.json",
Room.class);

imported.forEach(hotelService::addRoom);

System.out.println("Завантажено номерів: " +
imported.size());
                        } catch (IOException e) {

System.out.println("Помилка завантаження: " +
e.getMessage());
                        }
                        break;
```

```
                    default:

System.out.println("Невірний вибір. Оберіть
1-6.");
                }
            } catch (InputMismatchException e) {
                System.out.println("\nПОМИЛКА:
Вводьте лише ЧИСЛА!");
                scanner.nextLine();
            } catch (Exception e) {
                System.out.println("\nВиникла
помилка: " + e.getMessage());
            }
        }
    }

    private static void showStatistics(HotelService
service) {
        System.out.println("Заброньовано: " +
service.getOccupiedRoomsCount());
        List<Room> free =
service.getAvailableRoomsSorted();
        if (free.isEmpty()) {
            System.out.println("Вільні номери: [УСІ
ЗАЙНЯТІ]");
        } else {
            System.out.print("Вільні номери: ");
            free.forEach(r ->
System.out.print(r.getNumber() + " "));
            System.out.println();
```

```
        }

System.out.println("============================");
    }
}
```

3. Лістинг модульних тестів

Файл: src/test/java/com/hotel/**HotelProjectTest.java**.

```
package com.hotel;

import com.hotel.model.*;
import com.hotel.service.HotelService;
import com.hotel.service.JsonService; //ІМПОРТ
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.io.IOException;
import java.util.List;
import java.util.*;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class HotelProjectTest {
    private HotelService service;
    private Room room101;
    private Visitor visitor;

    @BeforeEach
    void setUp() {
```

```
        // Цей метод виконується перед КОЖНИМ тестом,
щоб дані були «свіжими»
        service = new HotelService();
        room101 = new Room(101, 500.0);
        visitor = new Visitor("Тетяна", "test@kpi.ua");
        service.addRoom(room101);
    }

    //ТЕСТИ ДЛЯ VISITOR

    @Test
    void testValidEmail() {
        assertTrue(visitor.isValidEmail(), "Email має
бути валідним");
    }

    @Test
    void testInvalidEmail() {
        Visitor badVisitor = new Visitor("Гість",
"bad@email");
        assertFalse(badVisitor.isValidEmail(), "Email
без крапки не має проходити валідацію");
    }

    //ТЕСТИ ДЛЯ ROOM

    @Test
    void testCalculateTotal() {
        assertEquals(1500.0, room101.calculateTotal(3),
0.001);
```

```
    }

    @Test
    void testCalculateTotalException() {
        assertThrows(IllegalArgumentException.class, ()
-> room101.calculateTotal(-5));
    }

    // ТЕСТИ ДЛЯ HOTELSERVICE

    @Test
    void testSortingLogic() {
        service.addRoom(new Room(47, 200.0));
        List<Room> sorted =
service.getAvailableRoomsSorted();
        assertEquals(47, sorted.get(0).getNumber(),
"Першим у списку має бути номер 47");
    }

    @Test
    void testBookingProcess() {
        service.bookRoom(visitor, 101, 2);
        assertFalse(room101.isAvailable(), "Кімната має
змінити статус на 'зайнята'");
    }

    @Test
    void testBookingOccupiedRoom() {
        service.bookRoom(visitor, 101, 2);
```

```
        assertThrows(NoSuchElementException.class, ()
-> service.bookRoom(visitor, 101, 1));
    }

    @Test
    void testReleaseRoomLogic() {
        service.bookRoom(visitor, 101, 2);
        service.releaseRoom(101);
        assertTrue(room101.isAvailable());

assertTrue(service.getBookingsSortedByRoom().isEmpty()
, "Список бронювань має стати порожнім");
    }

    @Test
    void testOccupiedCounter() {
        service.addRoom(new Room(102, 1000.0));
        service.bookRoom(visitor, 101, 1);
        assertEquals(1,
service.getOccupiedRoomsCount());
    }

    //Мок-тест: взаємодію без реального запису файлу
    @Test
    void testExportWithMock() throws IOException {
        JsonService mockJson = mock(JsonService.class);
        List<Room> testRooms = List.of(new Room(99,
100.0));

        mockJson.saveToFile("test.json", testRooms);
```

```
        verify(mockJson,
times(1)).saveToFile("test.json", testRooms);
    }
}
```

Результати виконання юніт-тестів:

HotelProjectTest (com.hotel)	1 sec 403 ms
testBookingOccupiedRoom()	43 ms
testValidEmail()	4 ms
testCalculateTotalException()	4 ms
testOccupiedCounter()	4 ms
testBookingProcess()	3 ms
testSortingLogic()	5 ms
testExportWithMock()	1 sec 333 ms
testInvalidEmail()	2 ms
testReleaseRoomLogic()	4 ms
testCalculateTotal()	1 ms

4. Приклад файлу з заекспортованими даними

Файл: rooms_data.json

```
[ {
 "number" : 47,
 "pricePerNight" : 200.0,
 "available" : true
}, {
 "number" : 75,
 "pricePerNight" : 100.0,
 "available" : true
}, {
 "number" : 102,
```

```
 "pricePerNight" : 1200.0,
 "available" : true
}, {
 "number" : 103,
 "pricePerNight" : 800.0,
 "available" : true
} ]
```


5. Скриншоти роботи застосунку

```
=== ГОТЕЛЬ 'Star Rain' ===
Заброньовано: 0
Вільні номери: 47 75 101 102 103
============================

--- МЕНЮ ---
1. Забронювати номер
2. Звільнити номер
3. Переглянути всі бронювання
4. Вийти
5. Збереження
6.Завантаження
Оберіть дію:
```

Рис.2 - Головне меню

```
Доступні номери (відсортовано):
Номер 47 | Ціна: $200.0 | Вільний: true
Номер 75 | Ціна: $100.0 | Вільний: true
Номер 101 | Ціна: $500.0 | Вільний: true
Номер 102 | Ціна: $1200.0 | Вільний: true
Номер 103 | Ціна: $800.0 | Вільний: true

Введіть номер кімнати: 101
Ваш Email: yourname@gmail.com
Повне ім'я: Тетяна
Реєструємо клієнта: Тетяна (ID: 1640)
Скільки ночей? 2
 Тетяна, заброньовано на 2 ночей.
Загальна сума: $1000.0
```

Рис.3 - Процес успішного бронювання

```
--- МЕНЮ ---
1. Забронювати номер
2. Звільнити номер
3. Переглянути всі бронювання
4. Вийти
5. Збереження
6.Завантаження
Оберіть дію: 5
Дані успішно збережено в rooms_data.json
```

Рис.4 - Повідомлення про успішне збереження в JSON

```
--- ДЕТАЛЬНИЙ ЗВІТ ---
Кімната №101 | Клієнт: Тетяна | Ночей: 2 | РАЗОМ: $1000.0

Загальна виручка готелю: $1000.0
```

Рис.5 - Звіт бронювань

Висновки

У ході виконання лабораторної роботи було розроблено консольний застосунок для автоматизації процесів готельного бізнесу. Проєкт реалізовано з дотриманням принципів об’єктно-орієнтованого програмування. Розділення коду на моделі (Room, Visitor, Booking) та сервіси (HotelService, JsonService) дозволило досягти високої декомпозиції та гнучкості системи. Впровадження модульного тестування за допомогою JUnit 5 забезпечило верифікацію ключових бізнес-методів (рахунок вартості, валідація даних). Використання бібліотеки Mockito дозволило протестувати шар збереження даних в ізоляції від файлової системи, що є стандартом у промисловій розробці. Реалізовано механізм серіалізації та десеріалізації об’єктів у формат JSON за допомогою бібліотеки Jackson. Це забезпечило збереження стану програми між сеансами роботи та можливість обміну даними.